

Efficient Data Storage Systems

(Sistemas de Armazenamento de Dados Eficientes)

Storage Optimizations

Part 1



Storage Optimizations

Landscape

● Performance

- Caching (*part 1*)
- Prefetching, read-ahead and indexing
- Tiering (*part 1*)
- I/O control policies (*part 1, annex*)
- RAID 0
- Data striping/sharding (*part 3*)

● Space management

- Compression (*part 2*)
- Deduplication (*part 2*)

● Security and privacy

- Encryption (*part 2*)
- Access control
- Trusted execution environments

● Redundancy and crash consistency

- Replication and erasure coding (*part 3*)
- RAID
- Journaling and write-ahead logging
- Checksumming
- **And many more!**

Caching

Overview

● Used to improve throughput and latency of “*applications*”

- Local, remote, and distributed environments
- Single vs. multi-tenant, synchronous vs. asynchronous
- Read, write, and mixed workloads
- In-memory, persistent, and hybrid

● Present across different levels of the storage stack

- Hardware-level (disk), kernel-level (page cache), user-level (applications)

● Challenges

- Cache coherence and invalidation (how to ensure synchronization?)
- Cache eviction and trashing (which data should be removed?)
- ...

Caching

Metrics

- To evaluate cache performance, one should consider several metrics
 - **Throughput:** total operations in a given time period (e.g., kops/s)
 - **(Tail) Latency:** (percentile) time taken per operation
 - **Hit rate:** percentage of operations serviced by the cache
 - **Miss rate:** percentage of times the cache could not serve a request
 - **Eviction rate:** determines how often data is removed due to space constraints

Caching

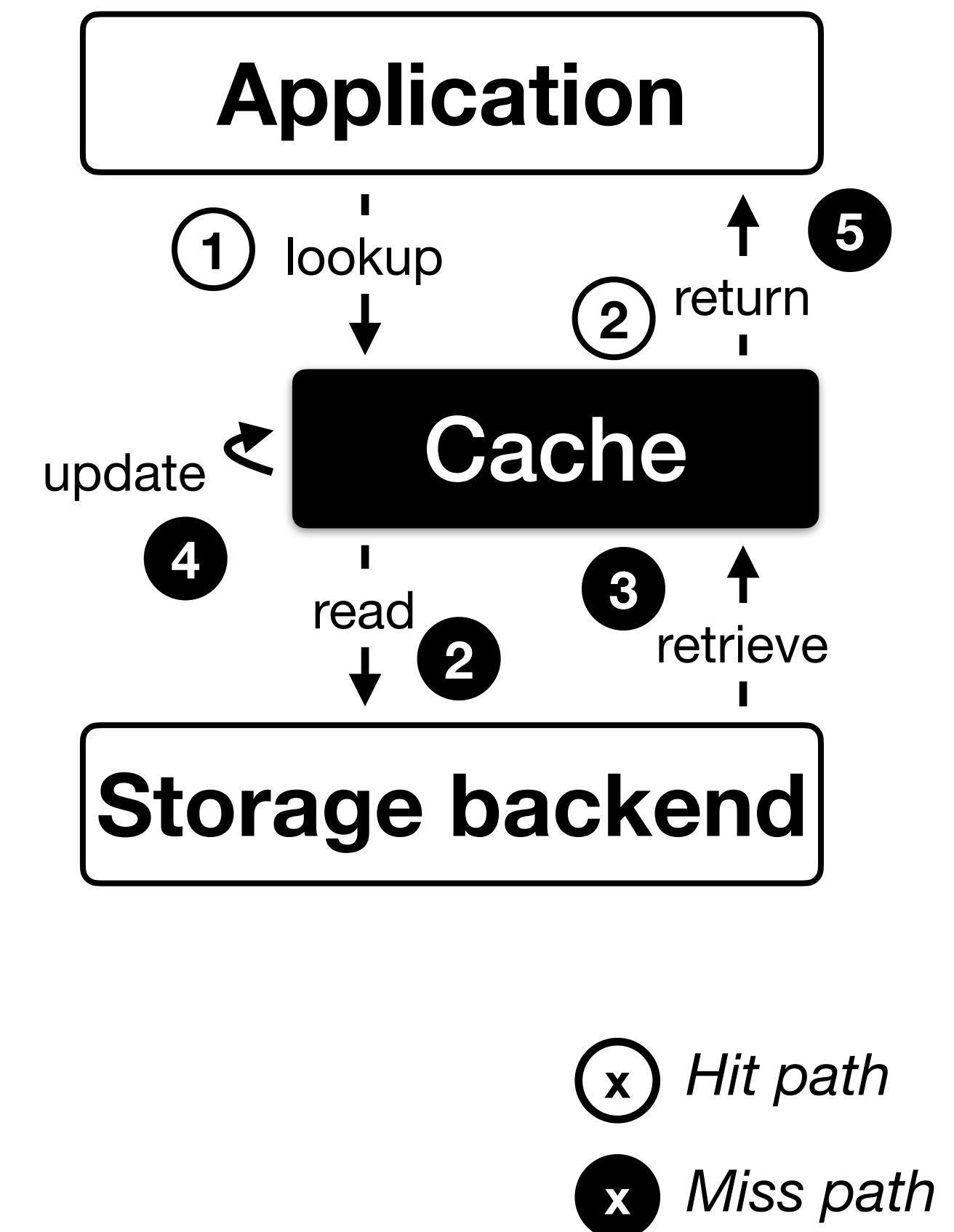
Caching strategies

- Caching strategies determine how data is read/written from/to the cache
- Read schemes
 - Read-through
 - Cache-aside
- Write schemes
 - Write-through
 - Write-back
 - Write-around
- Each caching scheme provides performance and durability trade-offs

Caching strategies

Read-through

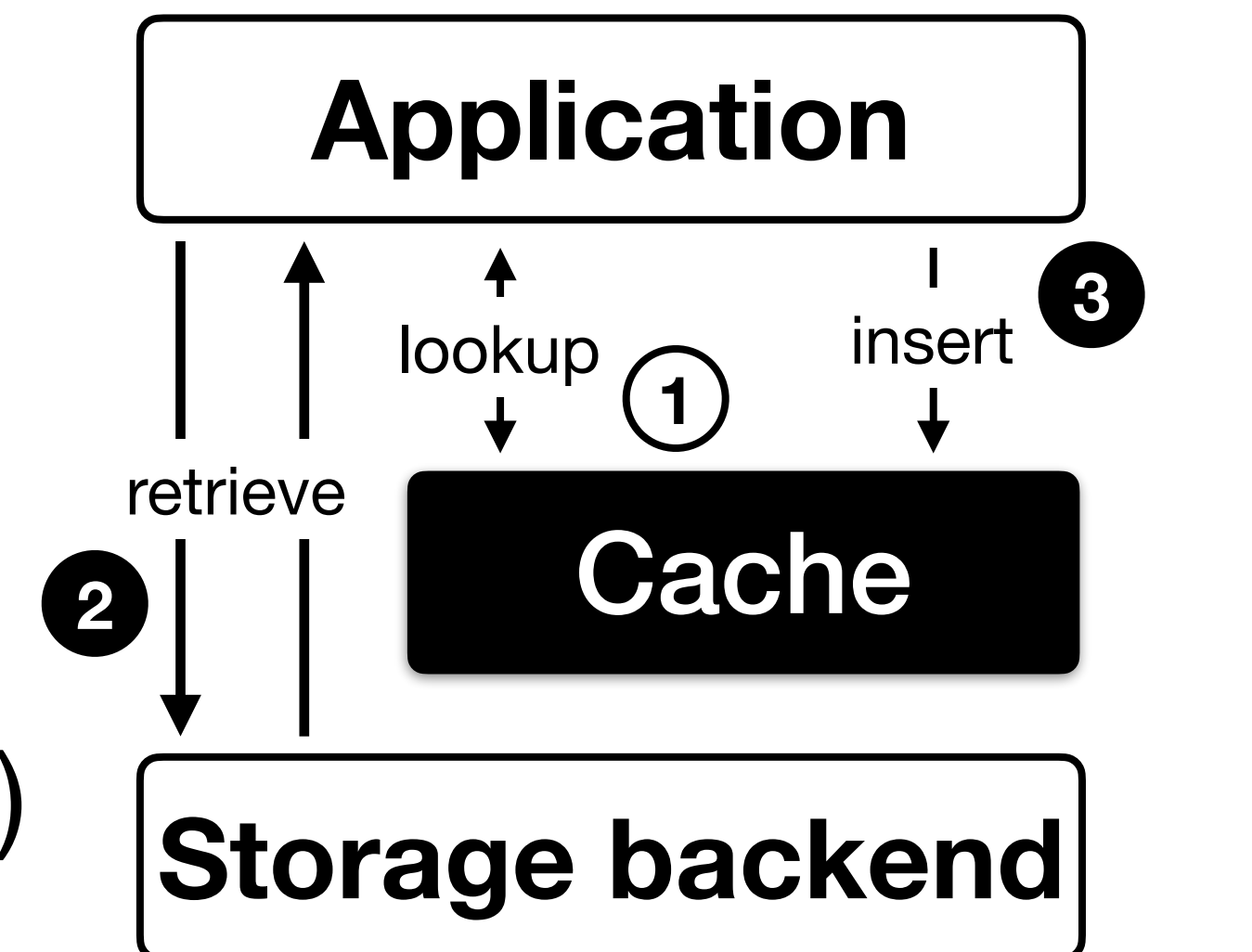
- All (read) requests are submitted to the cache, regardless of whether the data is in it
 - **Hit:** data is returned to the application
 - **Miss:** data is fetched from persistent storage, updated in the cache, and returned to the application
- Reduces cache misses and latency for frequently accessed data
- Introduces extra latency on first access
- Cache invalidation is required if not combined with a write-through caching scheme



Caching strategies

Cache-aside

- Read requests are first submitted to the cache, but the application may fetch data from persistent storage
 - ▶ **Hit:** data is returned to the application
 - ▶ **Miss:** data is read from the storage backend and inserted by the application in the cache
- Flexibility to the application (decide which data to cache)
- Cache misses cause high latency spikes
- Cache invalidation is required if not combined with a write-through caching scheme

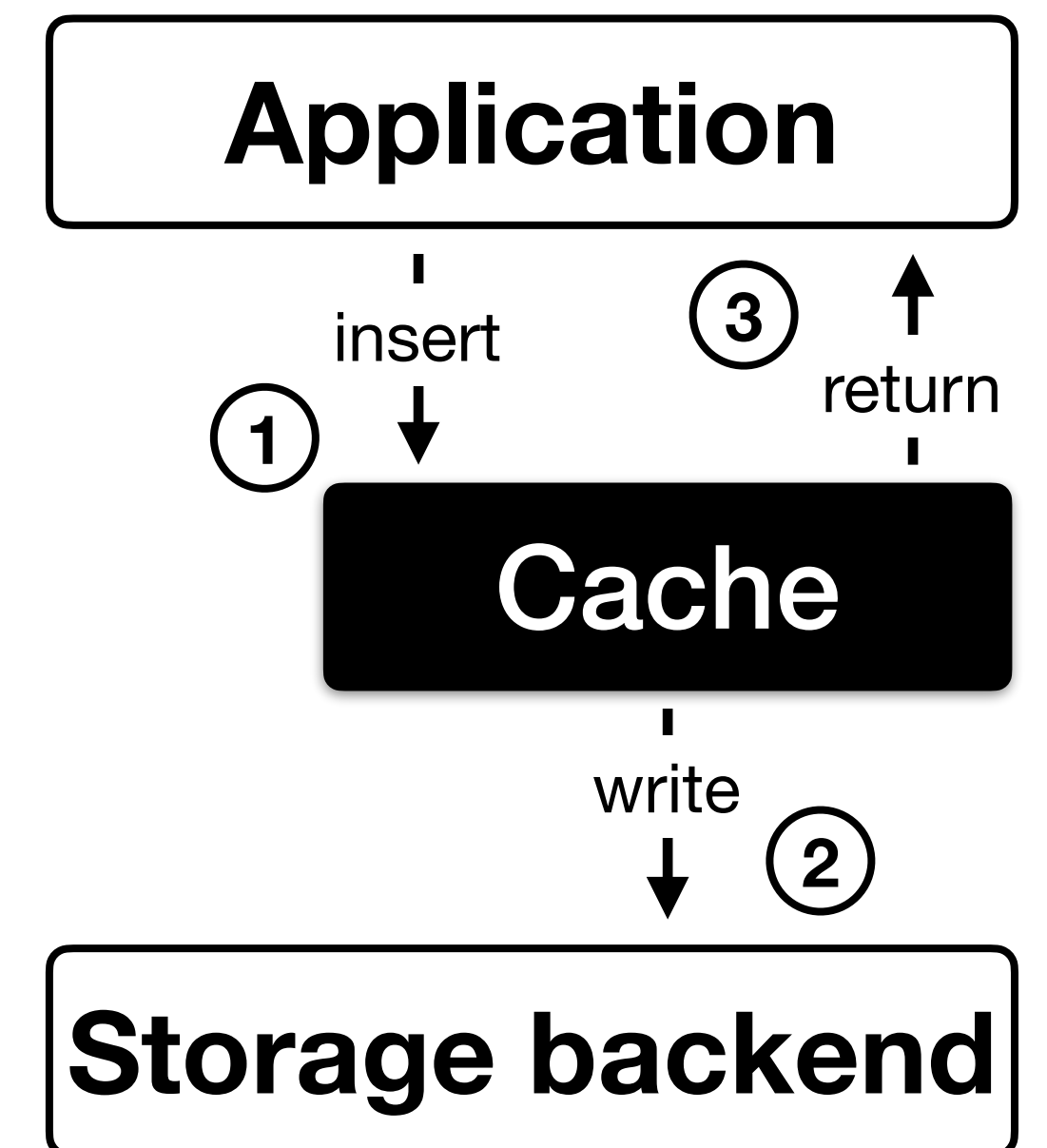


⊗ Hit path
⊗ Miss path

Caching strategies

Write-through

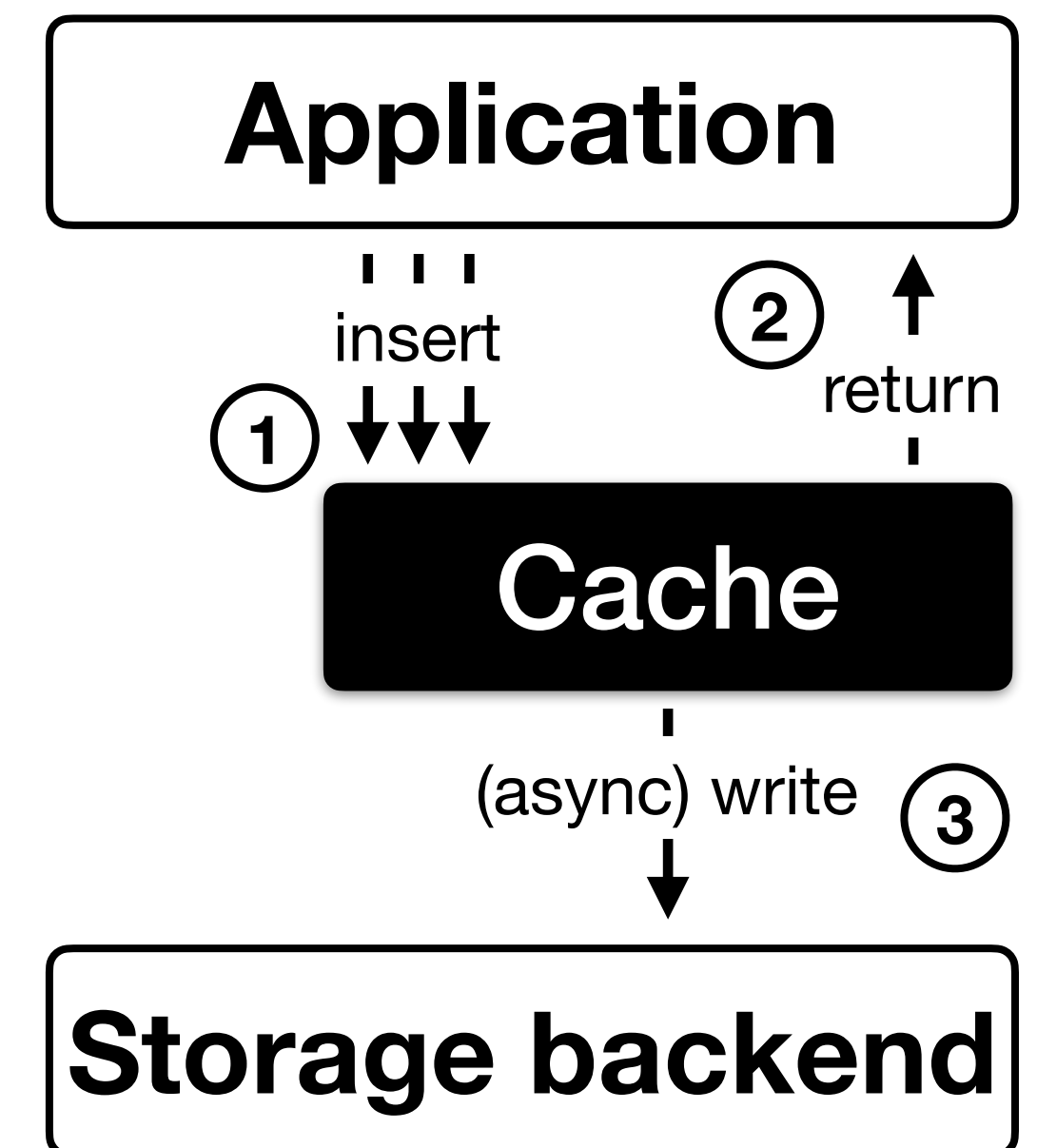
- Similarly to read-through, all (write) requests are submitted to the cache
- The cache synchronously updates the storage backend (blocking)
- Application resumes execution after both writes are made
- Stronger durability guarantees compared to write-back
- Automatically handles cache invalidation
- High write latency



Caching strategies

Write-back

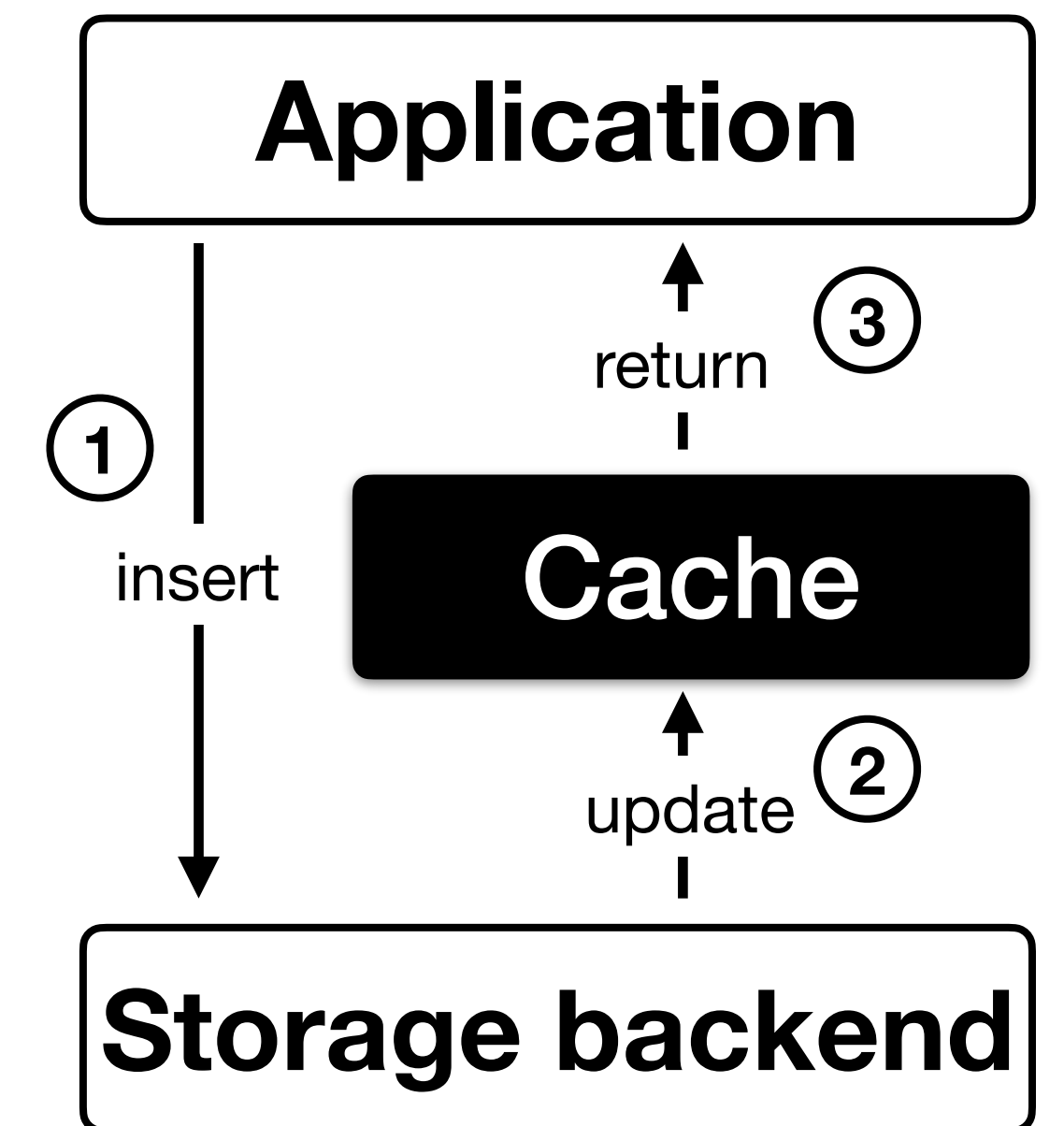
- All writes are submitted to the cache, being immediately acknowledged
- The cache asynchronously writes data to persistent storage
 - Single, larger requests (batching and coalescing)
 - Rate limiting policies to avoid saturating the backend
- High write throughput and low latency
- Low read latency for recently written data
- Risk of data loss if the cache fails before syncing



Caching strategies

Write-around

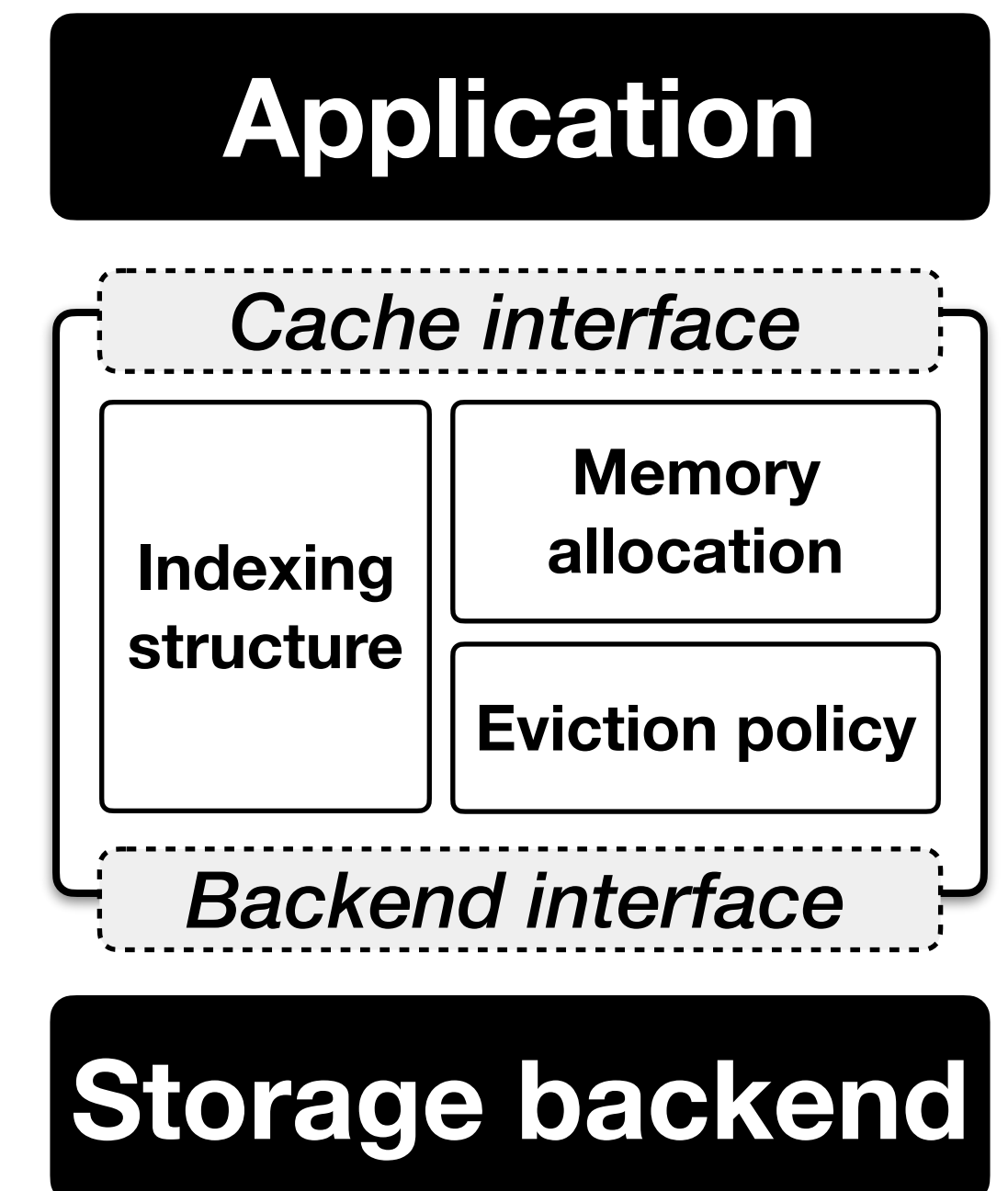
- Writes bypass the cache and go directly to the storage backend
- Data is only cached upon explicit request (e.g., read)
- Stronger durability guarantees when compared to other caching schemes
- Reduces unnecessary caching of rarely used data
- Higher read latency for recent writes
- Requires explicit cache invalidation



Caching systems

Design

- Cache and storage backend interfaces
 - Insert, lookup, ...
 - POSIX, io_uring, SPDK, ...
- Indexing structure
 - **Index:** B+tree, HashMap, ...
 - **Access *tracking*:** list-based, counters, ...
- Eviction policy
 - Least recently used (LRU), least frequently used (LFU), Random, TTL, Clock, ...
- Memory allocation

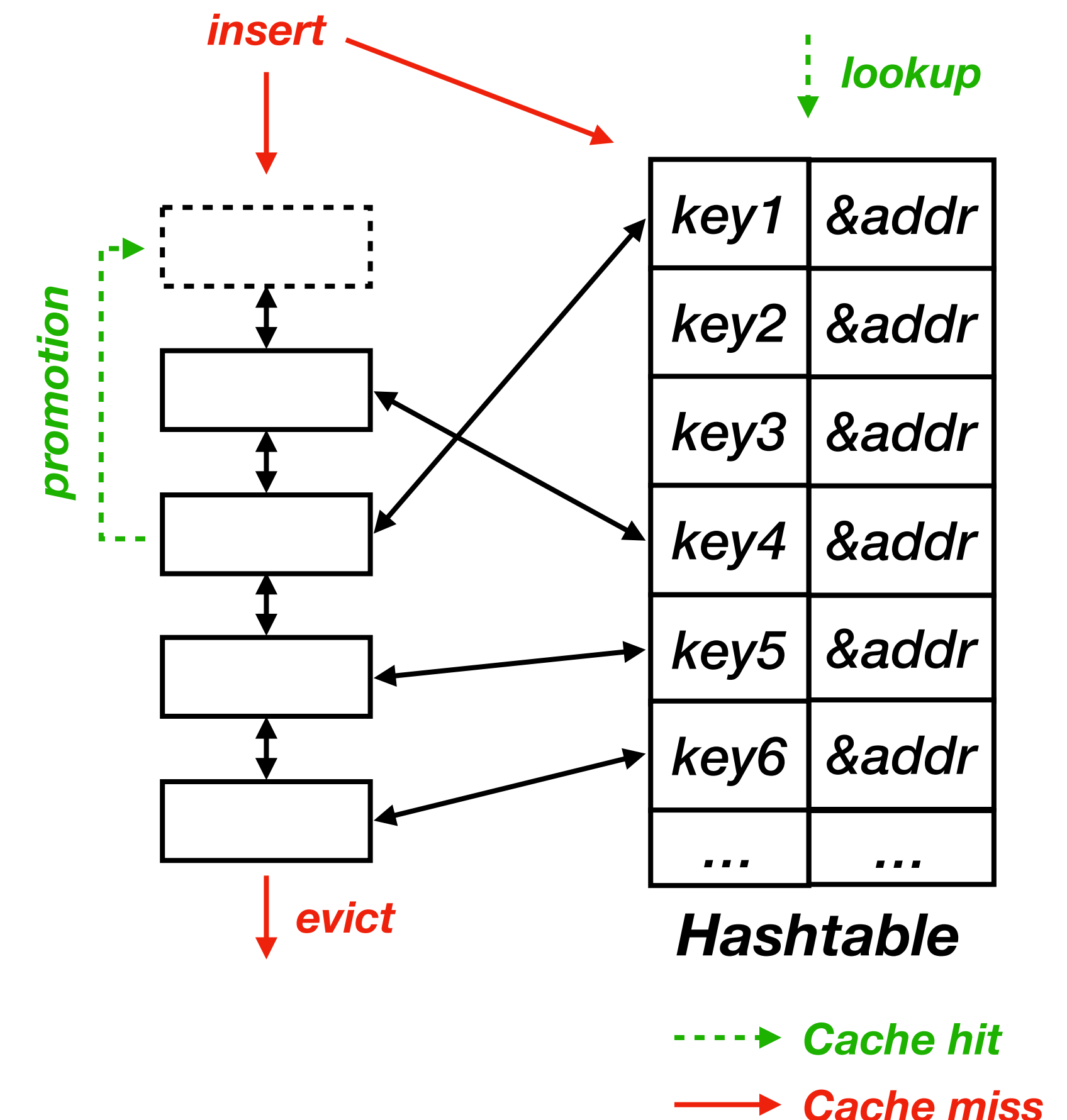


Caching systems

Indexing structure

General architecture of caches

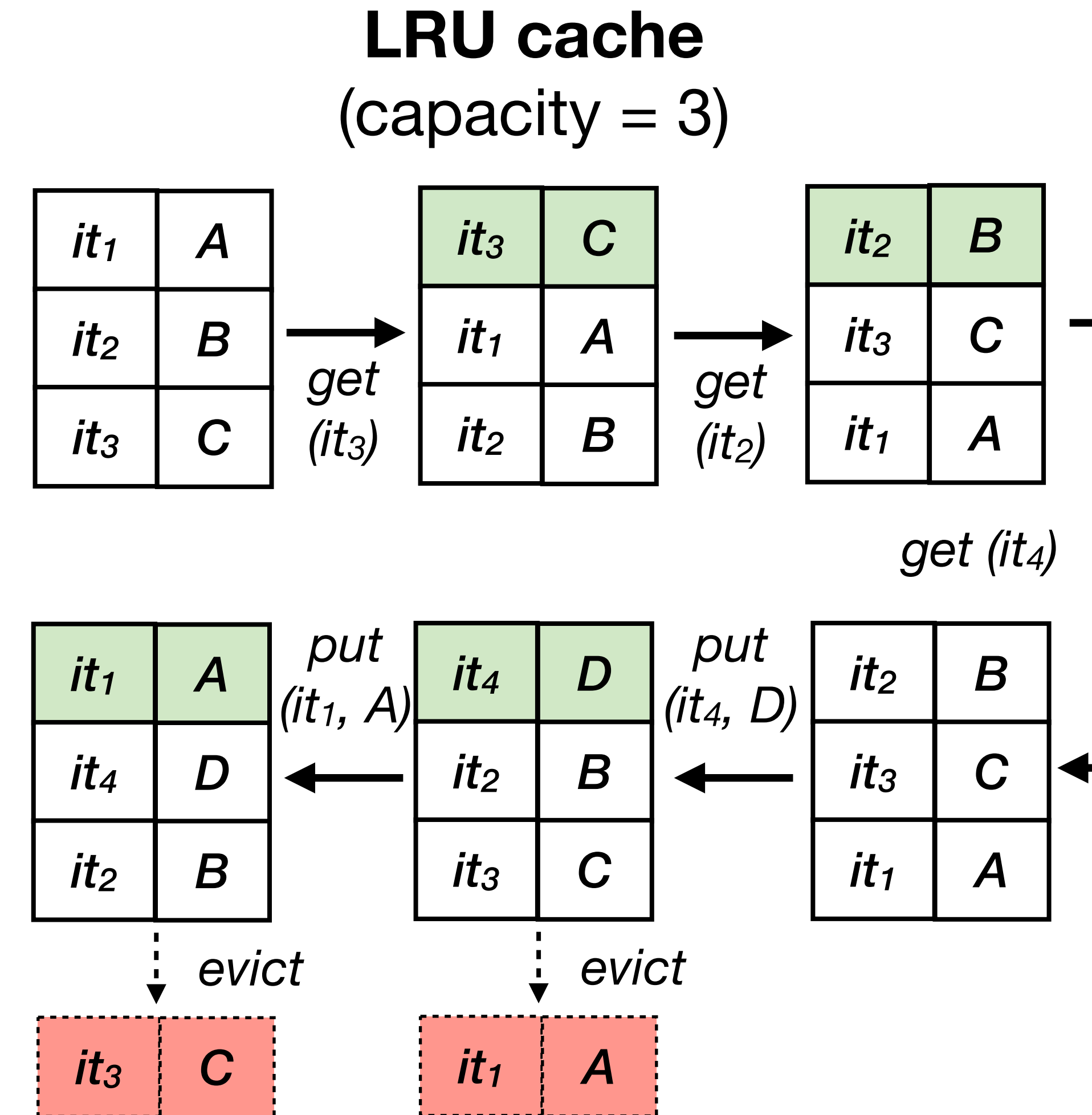
- **Index** maps keys and corresponding values
 - Enables fast data lookup
 - Values typically contain a pointer to the actual data (varies across implementations)
 - B+tree and HashMap
- **Access tracking** maintains the hotness of accessed items and manages cache evictions
- Both structures require thread-safety
- All operations require updates at least in one of the internal cache components



Caching systems

Eviction policies: Least Recently Used

- When the cache reaches its limit, it evicts the item that has been accessed the least recently
- Temporal locality:** items that have not been accessed for a longer period are less likely to be used in the near future
- Items are ordered with *timestamps**
- Wide adoption (databases, file systems, web caching)
- Ignores access frequency and susceptible to cache pollution (e.g., scan)

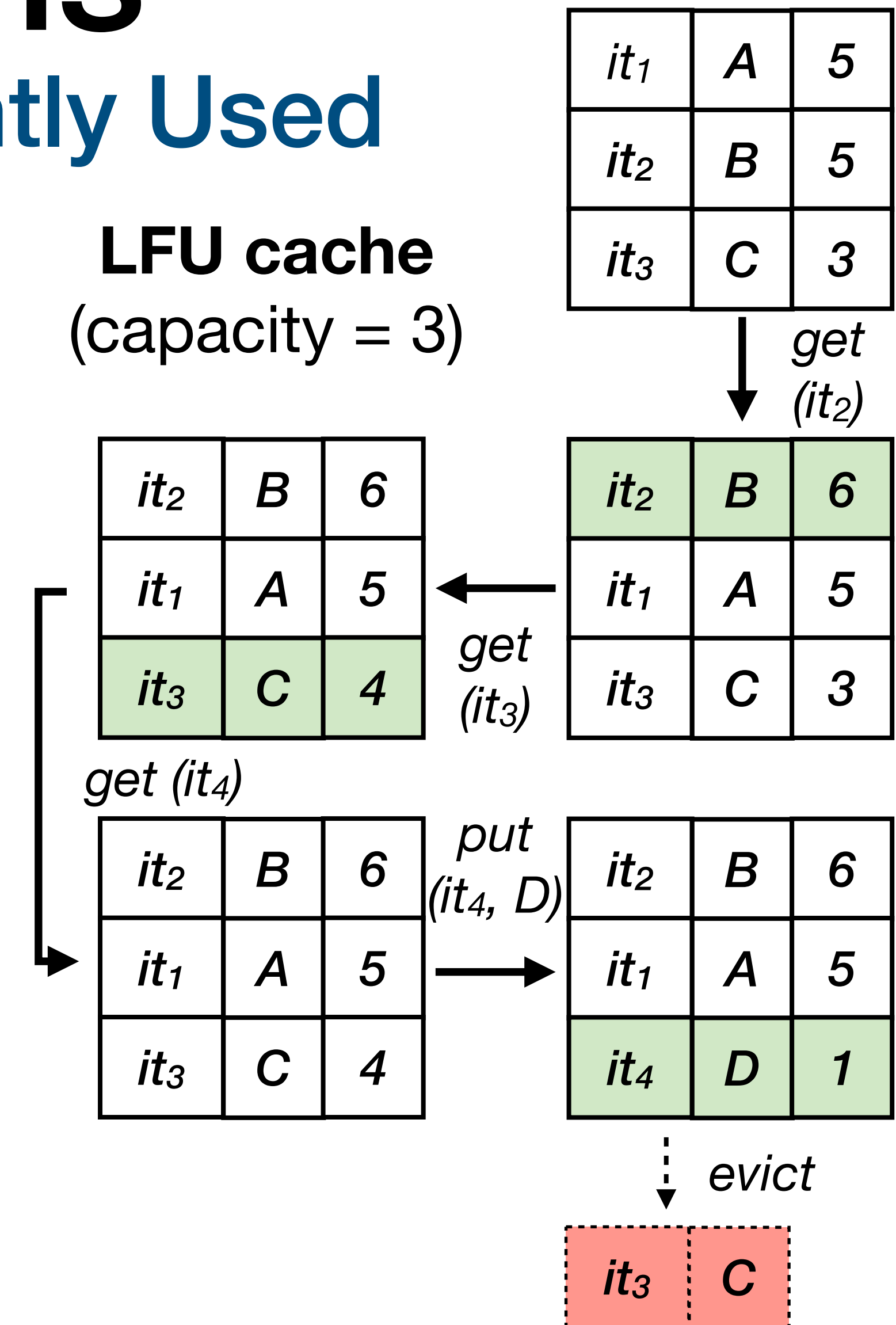


Caching systems

Eviction policies: Least Frequently Used

- When the cache reaches its limit, it evicts the item with the lowest access frequency
- *Access counters* to track the number of times each item is accessed
- Optimized for long-term trends (stable workloads)
- Reduces unnecessary evictions
- Slow to adapt to varying access patterns
- High memory overhead for access tracking
- Risk of stale data

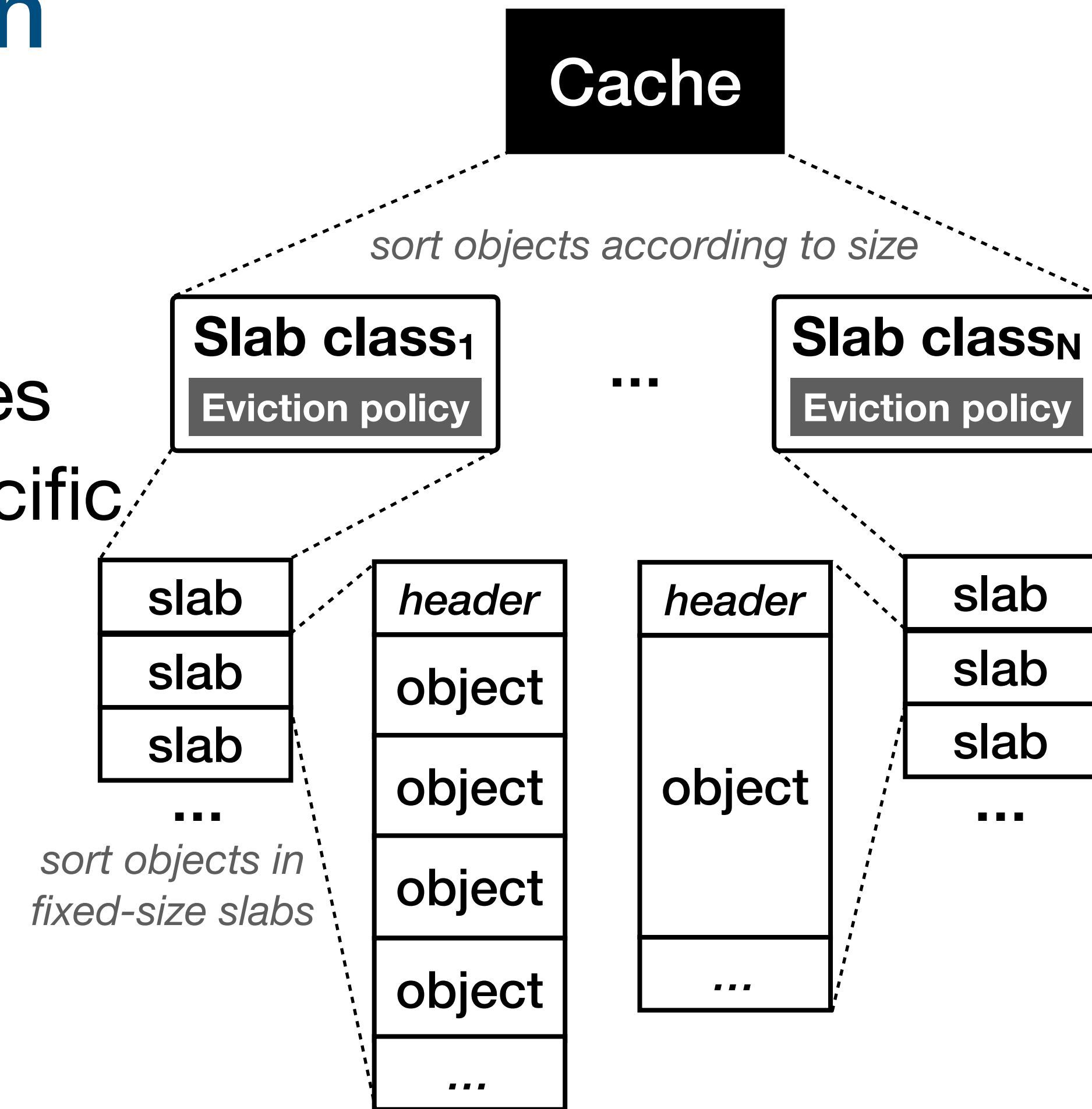
LFU cache
(capacity = 3)



Caching systems

Memory allocation

- Caches often implement a memory allocator
- *Slab-based allocation*
 - ▶ Memory is partitioned into different slab classes
 - ▶ Each slab class manages objects within a specific size range and a dedicated eviction scheme
 - ▶ Each slab contains multiple data items and an header with the items range
- *Log-based allocation*
 - ▶ Data items are appended in a contiguous log
 - ▶ Memory is accessed sequentially
 - ▶ Write-friendly allocation scheme

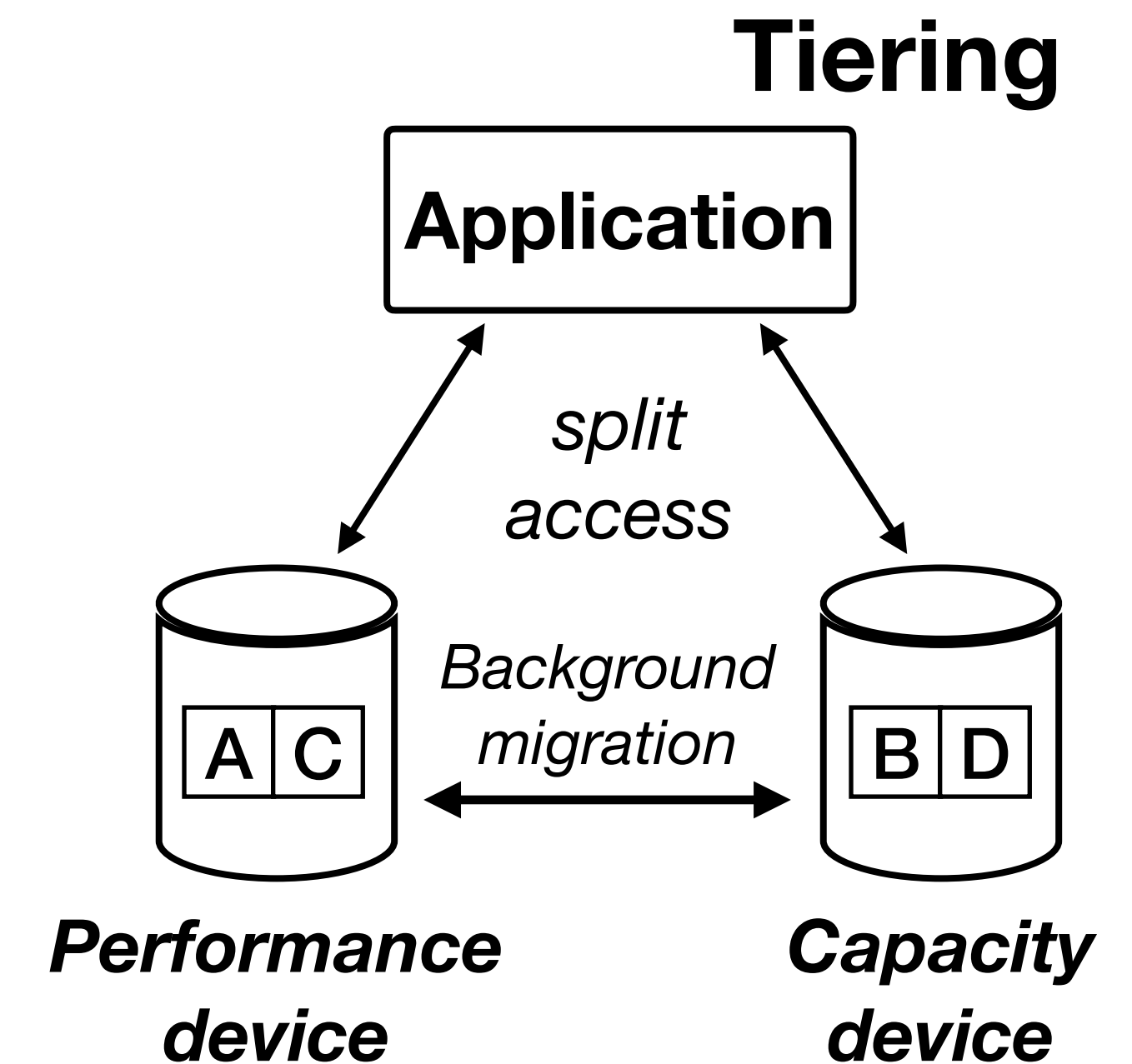
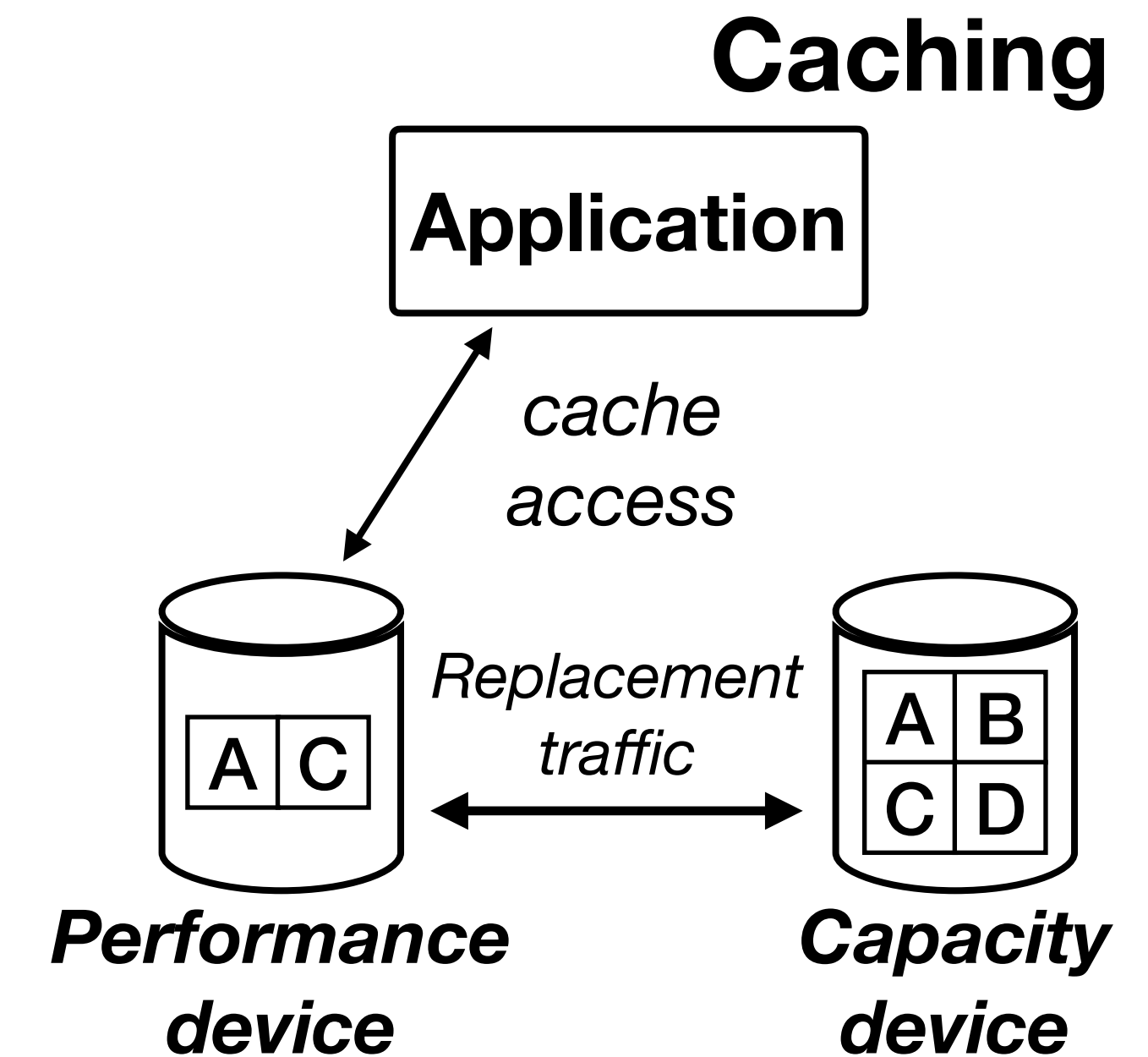


Example of slab-based allocation

Tiering

Overview

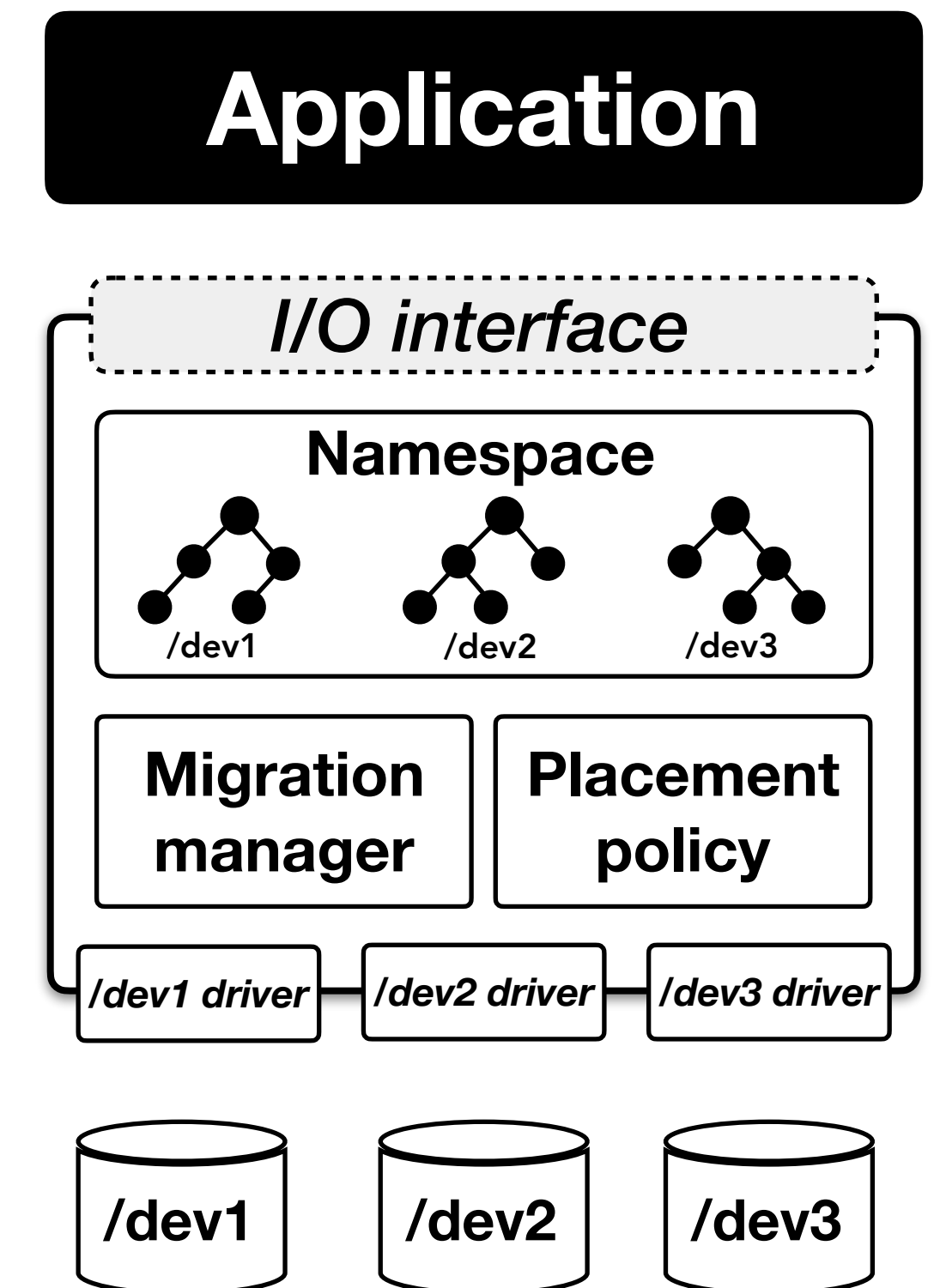
- *Caching* and *tiering* are often used to manage storage hierarchies made of heterogeneous devices
 - **Performance device:** fast, limited capacity, costly
 - **Capacity device:** slow but cheaper and larger
- In *caching*, hot data items are copied from the capacity to the performance device
 - Duplicated data across devices
- In *tiering*, data items are partitioned across devices according to a specific placement scheme
 - Items only reside in one of the devices



Tiering systems

Design

- I/O interface is typically preserved
 - POSIX, io_uring, SPDK, ...
- Unified **namespace**
 - Data and metadata management
 - Transparent request handling
- **Placement policy**
 - Performance, capacity, storage endurance, parallelism, monetary cost, ...
- **Migration management**
 - Move data across devices according a specific metric



Tiering

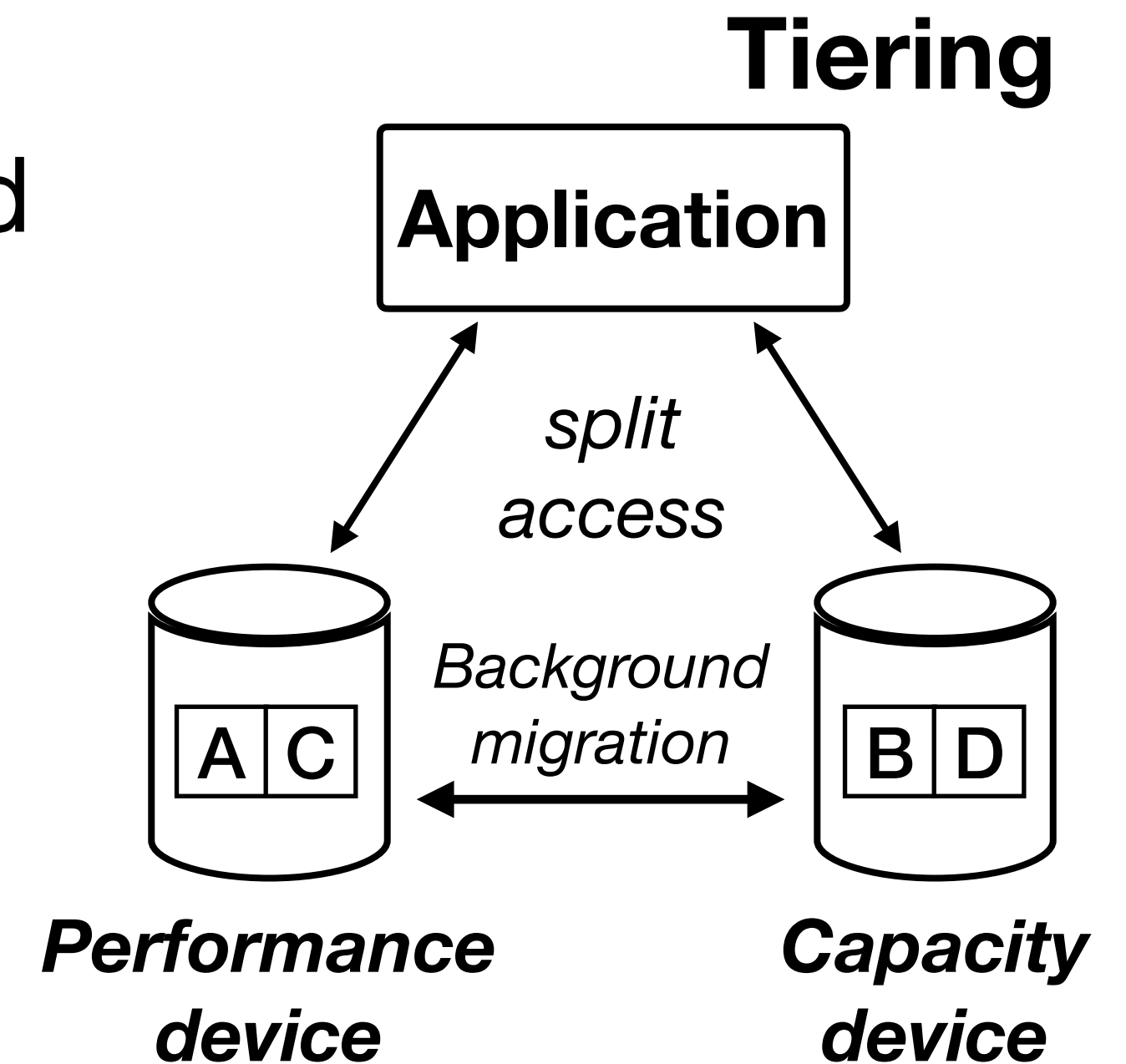
Placement schemes

● Static and policy-based tiering

- ▶ Data items are assigned to specific storage tiers based on predefined rules (file size, age, monetary cost, ...)
- ▶ Simple to implement
- ▶ Not adaptable to dynamic access patterns; risk of misplacing items

● Access frequency-based tiering

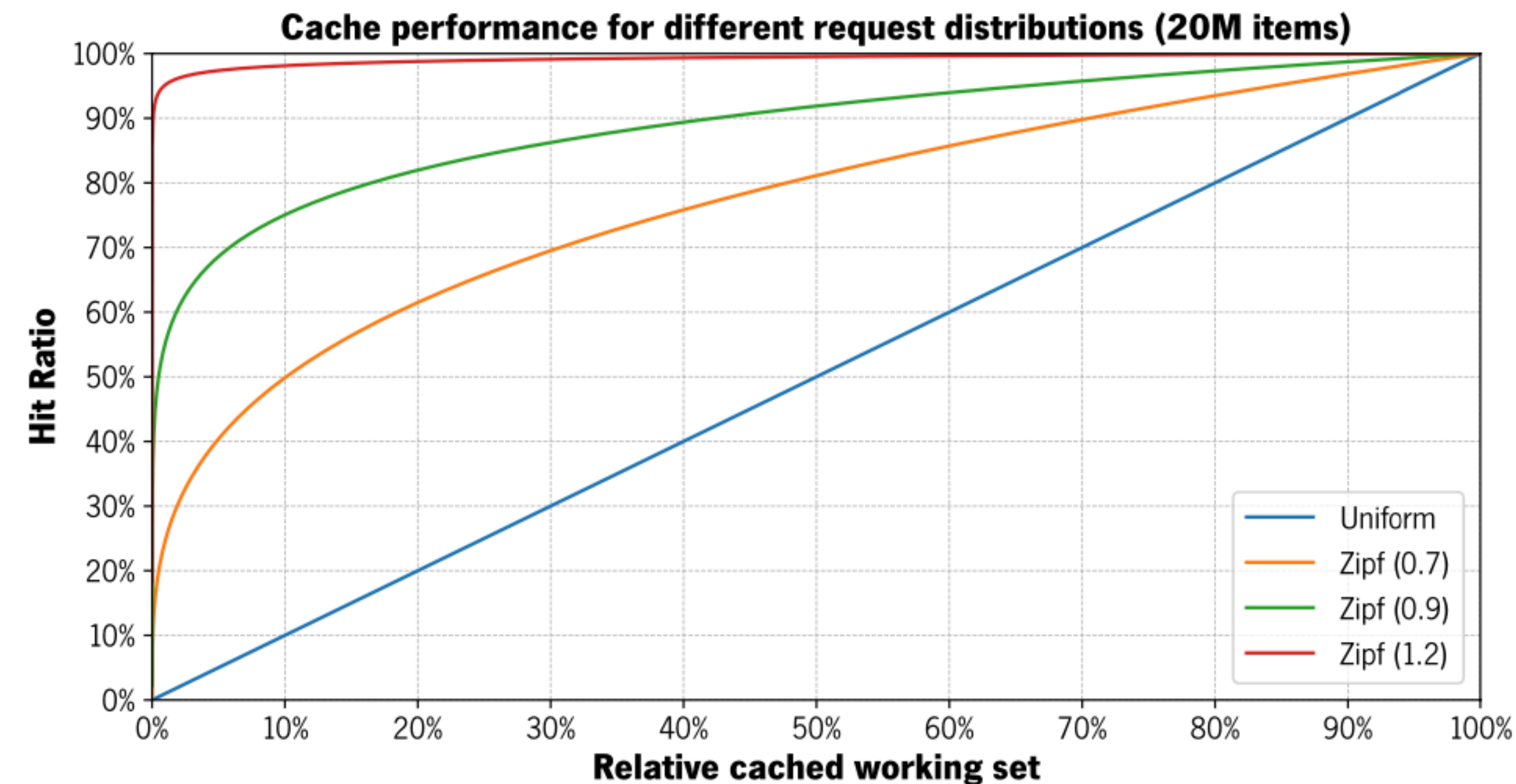
- ▶ Data items are moved between tiers based on access frequency (LFU)
- ▶ Dynamically adjusts to different access patterns
- ▶ Frequent tiering can cause performance overhead



Caching and tiering

Workload characteristics

- There are multiple factors that impact the effectiveness of caching and tiering schemes
- Operation proportion (read vs. write vs. scan)
- Request distribution (data popularity)
 - Uniform, sequential, zipfian
- Object size distribution
- Time-to-live (expiration date)
- **And many more!**



References

- © Andrea Arpaci-Dusseau and Remzi Arpaci-Dusseau. **Operating Systems: Three Easy Pieces**. Arpaci-Dusseau Books, 2018.
- © B. Berg et al. **The CacheLib caching engine: Design and experiences at scale**. *USENIX OSDI*, 2020.
- © K. Wu et al. **The storage hierarchy is not a hierarchy: Optimizing caching on modern storage devices with Orthus**. *USENIX FAST*, 2021.
- © R. Macedo et al. **Taming Metadata-intensive HPC Jobs Through Dynamic, Application-agnostic QoS Control**. *IEEE/ACM CCGrid*, 2023.
- © Z. Qiu et al. **FrozenHot Cache: Rethinking Cache Management for Modern Hardware**. *ACM EuroSys*, 2023.
- © R. Adão et al. **Keigo: Co-designing Log-structured Merge Key-Value Stores with a Non-Volatile, Concurrency-aware Storage Hierarchy**. *VLDB*, 2025.

I/O control policies

Overview

- I/O control policies enable ensuring storage quality-of-service (QoS) guarantees for applications that compete for shared storage resources
- Control mechanisms
 - Building blocks used for rate limiting I/O requests (data and metadata)
 - Token buckets, priority queues, I/O schedulers (block I/O controller)
- Control policies
 - Uniform
 - Priority-based rate limiting
 - Proportional sharing
 - ...

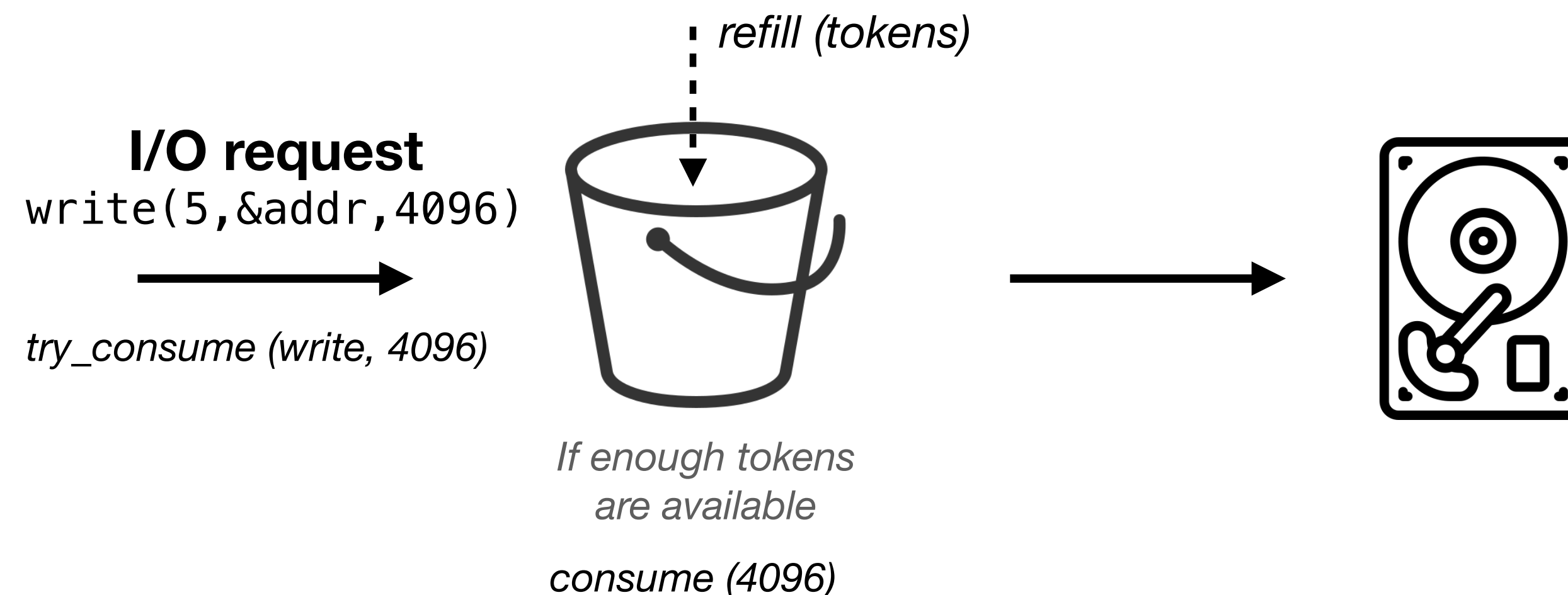
I/O control policies

Token bucket

- Mechanism used for controlling the rate of I/O workflows
 - Token bucket, leaky bucket, hierarchical token bucket, ...
- Prevents I/O burstiness and variability

Token Bucket

- Fixed capacity
- Fixed refill period



I/O control policies

Control policies

- Static control policies specify fixed I/O bandwidth (or IOPS) limits
- **Uniform rate distribution**
 - Disk bandwidth is evenly distributed across applications
 - Equally distributes resource shares among applications, but does not consider workload characteristics (access patterns, I/O load, ...)
- **Priority-based rate distribution**
 - Disk bandwidth is distributed based on a given priority
 - Applications with higher priority have access to a larger resource share
- Both policies cannot be dynamically adjusted, causing *under-* and *over-provisioning*

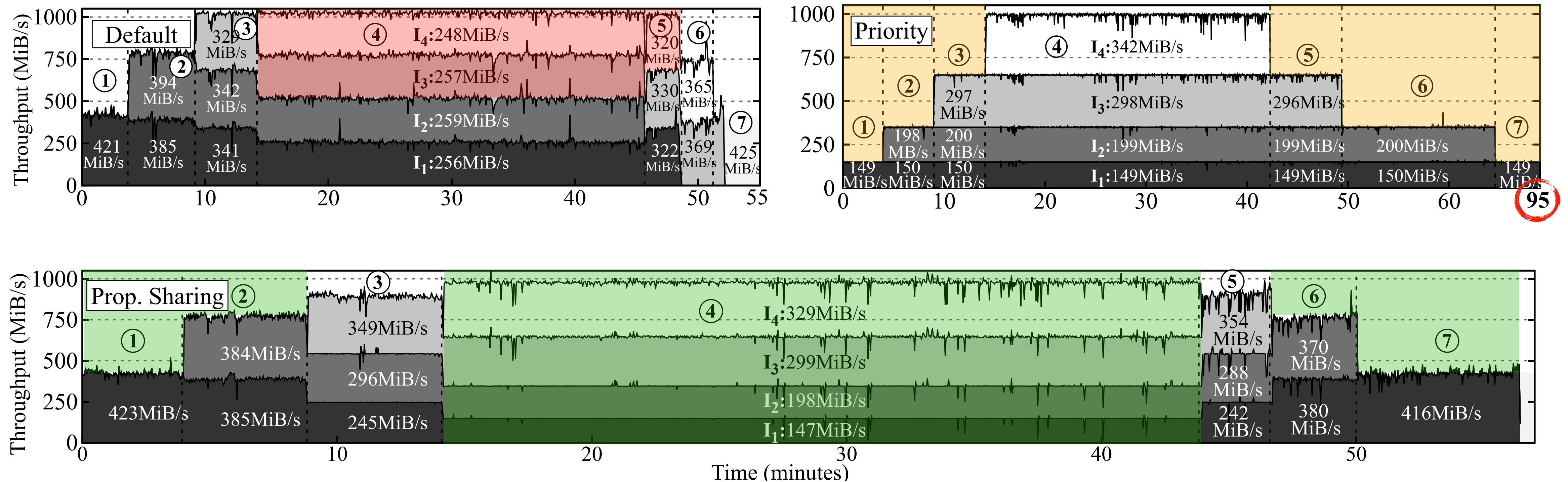
I/O control policies

Control policies

- Dynamic control policies allow assigning I/O limits that change over time
- Adaptable to workload and system variations
- **Proportional sharing**
 - Disk bandwidth is initially distributed across applications based on priority
 - No application gets a share larger than its demand
 - Jobs with unsatisfied demands get equal shares of resources
 - Whenever unused bandwidth is available, it is proportionately distributed among applications
- Prevents *under-provisioning* (and over-provisioning to some degree)

I/O control policies

Example: per-application bandwidth control



Application 1 {150 MiB/s}

Application 2 {200 MiB/s}

Application 3 {300 MiB/s}

Application 4 {350 MiB/s}